

Tests avec Spring

Tests unitaires, de composants et d'intégration sur une application de gestion de **duplicatas d'impôts** — du repository au contrôleur REST, en passant par la sécurité.

FORMATION SPRING

JAVA · SPRING BOOT · JPA



Agenda de la formation

01

Le domaine métier

Présentation de l'application duplicatas d'impôts

03

Tests d'intégration

Scénarios bout-en-bout avec @SpringBootTest

02

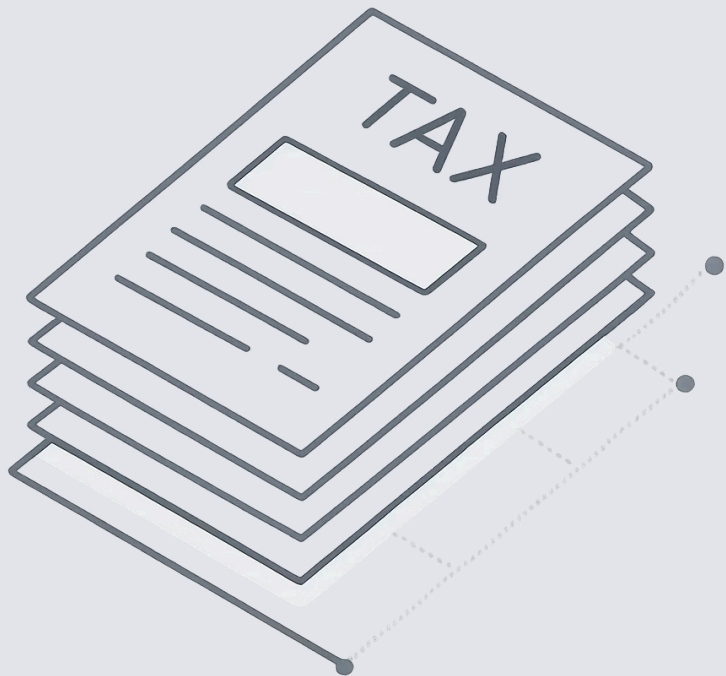
Tests de composants

Repository, Service, Contrôleur — couche par couche

04

Sécurité

Tester les rôles et les utilisateurs connectés

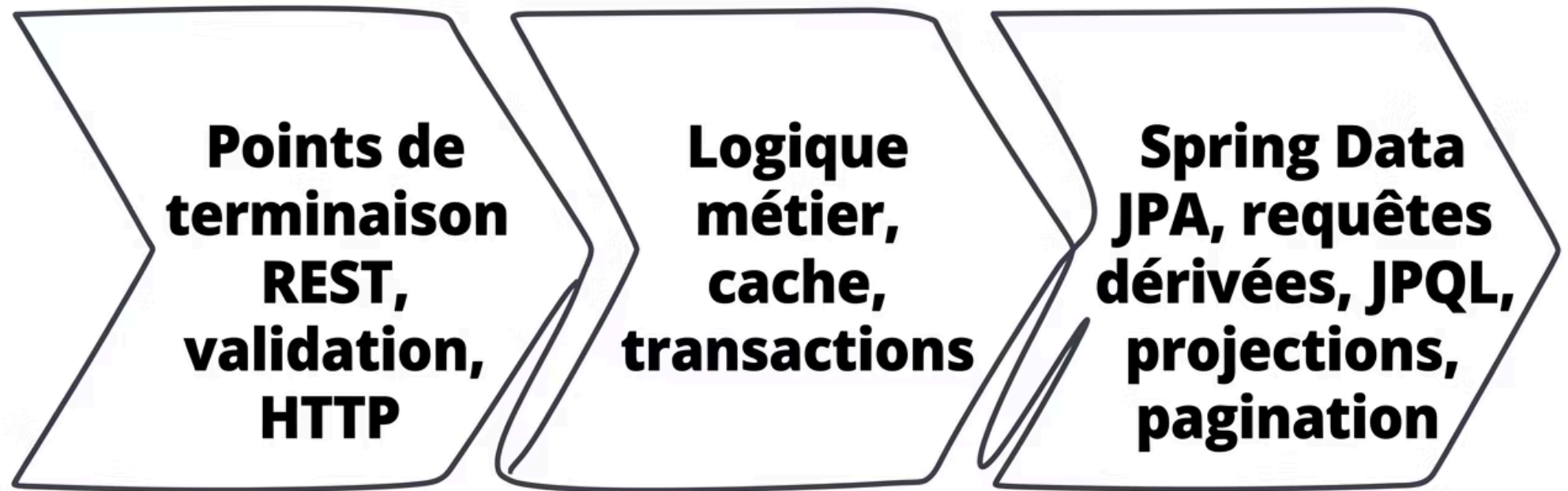


DOMAINE MÉTIER

L'application : Gestion de Duplicatas d'Impôts

Un duplicata est une copie officielle d'un avis d'imposition. L'application permet de **créer, consulter, rechercher et supprimer** des duplicatas associés à un identifiant fiscal utilisateur (`userId`) et un montant.

Architecture de l'application



Chaque couche a ses propres responsabilités et sera testée avec les outils adaptés : **MockMvc** pour le contrôleur, **Mockito** pour le service, **@DataJpaTest** pour le repository.

Controller

DuplicataControlleur

Service

DuplicataService

Repository

DuplicataRepository

Le contrôleur REST – Endpoints disponibles

Méthode	URL	Description
GET	/duplicatas	Lister tous les duplicatas
GET	/duplicatas/{id}	Visualiser un duplicata
POST	/duplicatas	Créer via paramètres query
POST	/duplicatas/{userId}/{montant}	Créer via @PathVariable
POST	/duplicatas_dto	Créer via corps JSON (@RequestBody)
DELETE	/duplicatas/{id}	Supprimer un duplicata
GET	/duplicatas/page	Recherche paginée et triée

Le Service – Responsabilités clés

Cache

@Cacheable et
@CacheEvict sur chaque
méthode

Transactions

@Transactional et
readOnly selon
l'opération

Validation métier

Vérifie l'existence de
l'utilisateur avant
création

Pagination

Contrôle les critères :
page, taille, tri, direction

```
public Duplicata createDuplicata(  
    String userId, int montant) {  
    User user = userService.findById(userId);  
    if (user == null)  
        throw new UserNotFoundException(userId);  
    Duplicata d = new Duplicata();  
    d.setUserId(userId);  
    d.setMontant(montant);  
    d.setPdfUrl(cdnUrl + "/pdfs/dummy.pdf");  
    return duplicataRepository.save(d);  
}
```

Le Repository – Spring Data JPA

Le repository étend `JpaRepository<Duplicata, String>` et déclare six types de requêtes pédagogiques :

1

Requête dérivée

`findById`

2

Between + OrderBy

`findByMontantBetweenOrderByMontantDesc`

3

ContainingIgnoreCase

Recherche textuelle insensible à la casse

4

JPQL @Query

Requête sur l'entité Java avec `:montantMinimum`

5

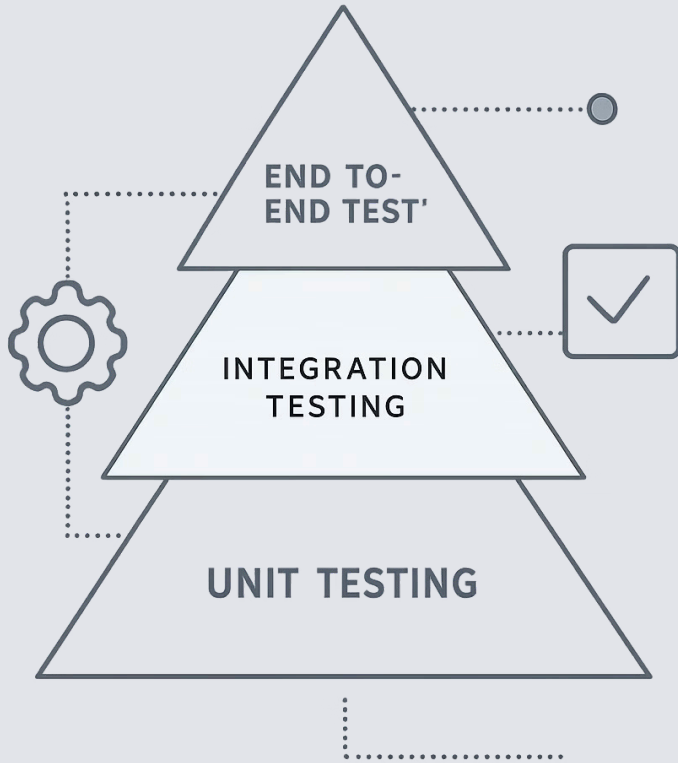
Projection

Vue partielle via `DuplicataResumeProjection`

6

Pageable

Pagination et tri dynamiques



STRATÉGIE DE TEST

La pyramide des tests

Nous suivrons la pyramide classique : de nombreux **tests unitaires** rapides à la base, des **tests de composants** ciblés au milieu, et des **tests d'intégration** complets au sommet.

Types de tests – Vue d'ensemble

Tests Unitaires

Testent une classe isolée, sans Spring.
Utilisent **Mockito** pour simuler les dépendances. Très rapides.

Tests de Composants

Chargent un slice du contexte Spring
: **@WebMvcTest**, **@DataJpaTest**.
Ciblés et efficaces.

Tests d'Intégration

Démarrent le contexte complet avec
@SpringBootTest. Testent le flux de bout en bout.

Tester le Repository avec @DataJpaTest

@DataJpaTest charge uniquement le contexte JPA : entités, repositories et une base H2 en mémoire. Aucun service ni contrôleur n'est instancié.

 Chaque test s'exécute dans une transaction annulée automatiquement : la base reste propre entre les tests.

```
@DataJpaTest
class DuplicataRepositoryTest {

    @Autowired
    DuplicataRepository repo;

    @Test
    void findByUserId_retourne_la_liste() {
        Duplicata d = new Duplicata();
        d.setUserId("123456789");
        d.setMontant(2500);
        repo.save(d);

        List<Duplicata> result =
            repo.findByUserId("123456789");

        assertThat(result).hasSize(1);
    }
}
```

Tester les requêtes dérivées

Les requêtes dérivées sont générées par Spring Data à partir du nom de la méthode. Il est important de les tester pour vérifier que le nom est bien interprété.

@Test

```
void findByMontantBetween_filtre_correctement() {  
    sauvegarder("user1", 1500);  
    sauvegarder("user1", 3000);  
    sauvegarder("user1", 6500);  
  
    List<Duplicata> result =  
        repo.findByMontantBetweenOrderByMontantDesc(  
            2000, 5000);  
  
    assertThat(result)  
        .hasSize(1)  
        .extracting(Duplicata::getMontant)  
        .containsExactly(3000);  
}
```

Ce que l'on vérifie

Le filtre d'intervalle est bien appliqué et le tri descendant est respecté.

Méthode helper

Factoriser la création de données dans une méthode privée `sauvegarder()` pour réduire la duplication.

Tester la requête JPQL et les Projections

@Test

```
void rechercherParMontantMinimum_jpql() {  
    sauvegarder("abc", 2000);  
    sauvegarder("abc", 4000);  
  
    List<Duplicata> result =  
        repo.rechercherParMontantMinimum(3000);  
  
    assertThat(result).hasSize(1);  
    assertThat(result.get(0).getMontant())  
        .isEqualTo(4000);  
}
```

@Test

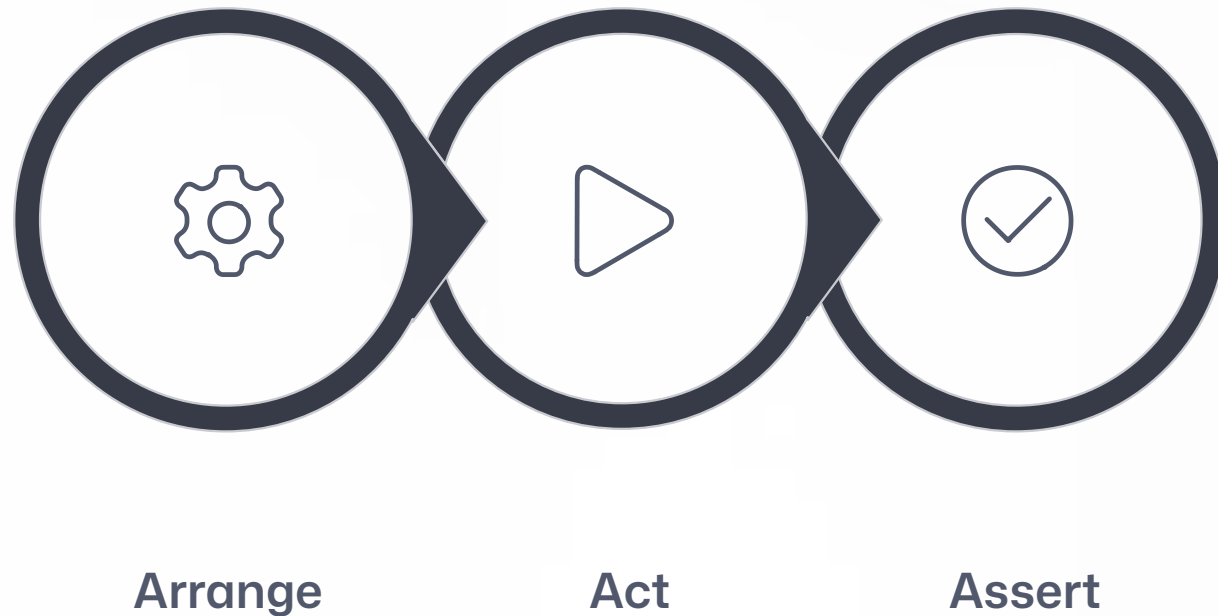
```
void listerProjections_retourne_vue_partielle() {  
    sauvegarder("xyz", 5000);  
  
    List<DuplicataResumeProjection> result =  
        repo.findByMontantGreaterThanOrEqualTo(1000);  
  
    assertThat(result).isNotEmpty();  
    assertThat(result.get(0).getUserId())  
        .isEqualTo("xyz");  
}
```

❏ La projection `DuplicataResumeProjection` est une interface Spring Data. Le test vérifie que seuls les champs déclarés sont exposés, sans charger l'entité complète.

Pour la pagination, injecter un `PageRequest` et vérifier le `Page<Duplicata>` retourné : contenu, nombre total de pages, taille.

Tester le Service en isolation

Le service contient la logique métier. On le teste **sans Spring**, en simulant toutes les dépendances avec **Mockito**.



Ce pattern **AAA — Arrange / Act / Assert** structure chaque test de façon lisible et maintenable.

Tester createDuplicata – Cas nominal

```
@ExtendWith(MockitoExtension.class)
class DuplicataServiceTest {

    @Mock DuplicataRepository repository;
    @Mock UserService userService;
    @InjectMocks DuplicataService service;

    @Test
    void createDuplicata_cas_nominal() {
        User user = new User("123456789", "Dupont");
        when(userService.findById("123456789"))
            .thenReturn(user);
        when(repository.save(any()))
            .thenReturn(i -> i.getArgument(0));

        Duplicata result =
            service.createDuplicata("123456789", 2500);

        assertThat(result.getUserId())
            .isEqualTo("123456789");
        assertThat(result.getMontant()).isEqualTo(2500);
        verify(repository).save(any(Duplicata.class));
    }
}
```

@Mock

Simule le repository et le userService — aucune base de données requise.

@InjectMocks

Injecte les mocks dans le service testé automatiquement.

verify()

Confirme que repository.save() a bien été appelé une fois.

Tester les cas d'erreur du Service

@Test

```
void createDuplicata_user_inconnu_leve_exception() {  
    when(userService.findById("inconnu"))  
        .thenReturn(null);  
  
    assertThatThrownBy(() ->  
        service.createDuplicata("inconnu", 2500))  
        .isInstanceOf(UserNotFoundException.class);  
  
    verify(repository, never())  
        .save(any());  
}
```

@Test

```
void getByld_introuvable_leve_exception() {  
    when(repository.findById("xxx"))  
        .thenReturn(Optional.empty());  
  
    assertThatThrownBy(() ->  
        service.getByld("xxx"))  
        .isInstanceOf(DuplicataNotFoundException.class);  
}
```



Toujours tester les cas d'erreur ! Un service robuste lève les bonnes exceptions avec les bons messages.

On vérifie aussi avec `verify(repository, never()).save(any())` que la sauvegarde n'est **pas appelée** en cas d'erreur.

Tester la validation de la pagination

```
@Test
```

```
void verifierCriteresPagination_page_negative() {  
    assertThatThrownBy(() ->  
        service.rechercherPagee("", -1, 5, "createdAt", "desc"))  
        .assertInstanceOf(InvalidSearchCriteriaException.class)  
        .hasMessageContaining("superieur ou egal a 0");  
}
```

```
@Test
```

```
void verifierCriteresPagination_tri_non_autorise() {  
    assertThatThrownBy(() ->  
        service.rechercherPagee("", 0, 5, "champInvalide", "desc"))  
        .assertInstanceOf(InvalidSearchCriteriaException.class)  
        .hasMessageContaining("Tri non autorise");  
}
```

❗ Les propriétés de tri autorisées sont : `createdAt`, `montant`, `userId`, `id`. Tester chaque violation de règle séparément garantit une couverture exhaustive.

Tester le Contrôleur avec @WebMvcTest

@WebMvcTest charge uniquement la couche web :
contrôleurs, filtres, converters. Le service est **mocké** avec
@MockBean.

i **MockMvc** simule des requêtes HTTP sans démarrer
un vrai serveur — rapide et précis.

```
@WebMvcTest(DuplicataControlleur.class)
class DuplicataControlleurTest {

    @Autowired MockMvc mockMvc;

    @MockBean DuplicataService service;

    @Test
    void getDuplicatas_retourne_200() throws Exception {
        when(service.getDuplicatas())
            .thenReturn(List.of(new Duplicata()));

        mockMvc.perform(get("/duplicatas"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$").isArray());
    }
}
```

Tester GET /duplicatas/{id}

@Test

```
void getDuplicata_trouve_retourne_200()
    throws Exception {
    Duplicata d = new Duplicata();
    d.setId("dup-001");
    d.setUserId("123456789");
    d.setMontant(2500);

    when(service.getById("dup-001")).thenReturn(d);

    mockMvc.perform(get("/duplicatas/dup-001"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value("dup-001"))
        .andExpect(jsonPath("$.montant").value(2500));
}
```

@Test

```
void getDuplicata_introuvable_retourne_404()
    throws Exception {
    when(service.getById("xxx"))
        .thenThrow(new DuplicataNotFoundException("xxx"));

    mockMvc.perform(get("/duplicatas/xxx"))
        .andExpect(status().isNotFound());
}
```

jsonPath()

Vérifie la structure et les valeurs du JSON retourné de façon expressive.

404 automatique

Spring MVC traduit `DuplicataNotFoundException` en HTTP 404 via un `@ExceptionHandler`.

Tester POST avec @RequestParam et @PathVariable

@Test

```
void createDuplicata_params_valides_retourne_200()
    throws Exception {
    Duplicata d = creerDuplicata("123456789", 2500);
    when(service.createDuplicata("123456789", 2500)).thenReturn(d);

    mockMvc.perform(post("/duplicatas")
        .param("user_id", "123456789")
        .param("montant", "2500"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.userId").value("123456789"));
}
```

@Test

```
void createDuplicata_montant_invalide_retourne_400()
    throws Exception {
    mockMvc.perform(post("/duplicatas")
        .param("user_id", "123456789")
        .param("montant", "500")) // < 1000 : invalide
        .andExpect(status().isBadRequest());
}
```

- ❏ La validation Bean Validation (@Min(1000)) est déclenchée par @Validated sur le contrôleur et gérée avant même d'appeler le service.

Tester POST avec @RequestBody (DTO JSON)

@Test

```
void createDuplicata_dto_valide_retourne_200()
    throws Exception {
    String json = ""
        {"user_id":"123456789","montant":2500}
        "";
    Duplicata d = creerDuplicata("123456789", 2500);
    when(service.createDuplicata("123456789", 2500))
        .thenReturn(d);

    mockMvc.perform(post("/duplicatas_dto")
        .contentType(MediaType.APPLICATION_JSON)
        .content(json))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.montant").value(2500));
}
```

@Test

```
void createDuplicata_dto_json_invalide_400()
    throws Exception {
    String json = ""
        {"user_id":"","montant":500}
        "";
    mockMvc.perform(post("/duplicatas_dto")
        .contentType(MediaType.APPLICATION_JSON)
        .content(json))
        .andExpect(status().isBadRequest());
}
```



Toujours préciser

`contentType(MediaType.APPLICATION_JSON)` pour les requêtes avec corps JSON, sinon Spring retourne un 415 Unsupported Media Type.



Tester les deux extrêmes : le cas valide **ET** le cas invalide. La couverture des erreurs est aussi importante que le cas nominal.

Tester DELETE /duplicatas/{id}

@Test

```
void deleteDuplicata_existant_retourne_204()
    throws Exception {
    doNothing().when(service).deleteById("dup-001");

    mockMvc.perform(delete("/duplicatas/dup-001"))
        .andExpect(status().isNoContent());

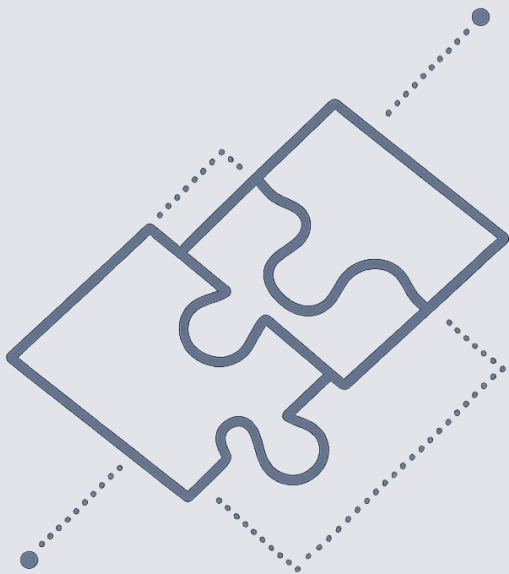
    verify(service).deleteById("dup-001");
}
```

@Test

```
void deleteDuplicata_introuvable_retourne_404()
    throws Exception {
    doThrow(new DuplicataNotFoundException("dup-999"))
        .when(service).deleteById("dup-999");

    mockMvc.perform(delete("/duplicatas/dup-999"))
        .andExpect(status().isNotFound());
}
```

❏ Pour les méthodes void, utilisez `doNothing().when()` et `doThrow().when()` plutôt que `when().thenReturn()`.



TESTS D'INTÉGRATION

@SpringBootTest – Le test bout-en-bout

@SpringBootTest démarre le contexte Spring complet avec la vraie base H2. On teste le flux entier : HTTP → Contrôleur → Service → Repository → Base de données.

Configuration du test d'intégration

```
@SpringBootTest(
    webEnvironment =
        SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
class DuplicataIntegrationTest {

    @Autowired MockMvc mockMvc;

    @Autowired DuplicataRepository repository;

    @BeforeEach
    void setUp() {
        repository.deleteAll();
    }

    private Duplicata sauvegarder(
        String userId, int montant) {
        Duplicata d = new Duplicata();
        d.setUserId(userId);
        d.setMontant(montant);
        return repository.save(d);
    }
}
```

RANDOM_PORT

Démarre sur un port aléatoire pour éviter les conflits.

@AutoConfigureMockMvc

Injecte MockMvc configuré avec le contexte complet.

@BeforeEach

Nettoie la base avant chaque test pour garantir l'isolation.

Scénario d'intégration complet

```
@Test
void flux_complet_creation_lecture_suppression()
throws Exception {
    // 1. Créer
    mockMvc.perform(post("/duplicatas")
        .param("user_id", "123456789")
        .param("montant", "3000"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").exists());

    // 2. Lister — doit contenir 1 élément
    mockMvc.perform(get("/duplicatas"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$", hasSize(1)));

    // 3. Récupérer l'id et supprimer
    String id = repository.findAll().get(0).getId();
    mockMvc.perform(delete("/duplicatas/" + id))
        .andExpect(status().isNoContent());

    // 4. Vérifier que la base est vide
    assertThat(repository.findAll()).isEmpty();
}
```

Tester la pagination en intégration

@Test

```
void rechercherPagee_retourne_page_correcte()
    throws Exception {
    sauvegarder("user1", 1500);
    sauvegarder("user1", 2500);
    sauvegarder("user2", 3500);

    mockMvc.perform(get("/duplicatas/page")
        .param("q", "user1")
        .param("page", "0")
        .param("size", "2")
        .param("sort", "montant")
        .param("direction", "asc"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.content", hasSize(2)))
        .andExpect(jsonPath("$.totalElements").value(2))
        .andExpect(jsonPath("$.content[0].montant")
            .value(1500));
}
```

On vérifie :

- Le contenu de la page (2 éléments filtrés sur "user1")
- Le total réel en base (totalElements)
- L'ordre de tri ascendant sur le montant

 Les tests d'intégration de pagination valident simultanément le service, le repository et la réponse HTTP.



SÉCURITÉ

Tester la sécurité avec Spring Security

Spring Security intercepte les requêtes avant qu'elles atteignent le contrôleur. Tester la sécurité, c'est vérifier que les **bons rôles** ont accès aux bons endpoints, et que les accès non autorisés sont bloqués.

Configuration des tests de sécurité

```
@WebMvcTest(DuplicataControlleur.class)
@Import(SecurityConfig.class)
class DuplicataSecuriteTest {

    @Autowired MockMvc mockMvc;
    @MockBean DuplicataService service;

    @Test
    @WithMockUser(roles = "USER")
    void user_peut_lister_les_duplicatas()
        throws Exception {
        when(service.getDuplicatas())
            .thenReturn(List.of());

        mockMvc.perform(get("/duplicatas"))
            .andExpect(status().isOk());
    }

    @Test
    void anonyme_recoit_401()
        throws Exception {
        mockMvc.perform(get("/duplicatas"))
            .andExpect(status().isUnauthorized());
    }
}
```

@WithMockUser

Simule un utilisateur authentifié avec le rôle spécifié, sans Spring Security complet.

@Import(SecurityConfig.class)

Charge la vraie configuration de sécurité pour tester les règles réelles.

Sans annotation

Aucun utilisateur injecté = requête anonyme → 401 Unauthorized.

Tester les rôles ADMIN vs USER

```
@Test
@WithMockUser(roles = "ADMIN")
void admin_peut_supprimer_un_duplicata()
throws Exception {
    doNothing().when(service).deleteById("dup-001");

    mockMvc.perform(delete("/duplicatas/dup-001"))
        .andExpect(status().isNoContent());
}
```

```
@Test
@WithMockUser(roles = "USER")
void user_ne_peut_pas_supprimer_un_duplicata()
throws Exception {
    mockMvc.perform(delete("/duplicatas/dup-001"))
        .andExpect(status().isForbidden()); // 403
}
```

```
@Test
@WithMockUser(username = "alice", roles = "USER")
void user_voit_seulement_ses_propres_duplicatas()
throws Exception {
    when(service.rechercherParUserId("alice"))
        .thenReturn(List.of());

    mockMvc.perform(get("/duplicatas/by-user/alice"))
        .andExpect(status().isOk());
}
```


Tester la sécurité en intégration

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@AutoConfigureMockMvc
class DuplicataSecuriteIntegrationTest {

    @Autowired MockMvc mockMvc;

    @Test
    @WithMockUser(roles = "ADMIN")
    void admin_acces_complet_au_flux()
        throws Exception {
        mockMvc.perform(post("/duplicatas")
            .param("user_id", "123456789")
            .param("montant", "2500"))
            .andExpect(status().isOk());
    }

    @Test
    @WithMockUser(roles = "USER")
    void user_interdit_sur_post_duplicata()
        throws Exception {
        mockMvc.perform(post("/duplicatas")
            .param("user_id", "123456789")
            .param("montant", "2500"))
            .andExpect(status().isForbidden());
    }
}
```

⚠ En tests d'intégration, la configuration de sécurité réelle est active. Ne pas la mocker — c'est précisément ce qu'on veut tester.

Le tableau des codes HTTP attendus selon le rôle :

Rôle	GET	POST	DELETE
Anonyme	401	401	401
USER	200	403	403
ADMIN	200	200	204

Bonnes pratiques – Récapitulatif



Une couche à la fois

@DataJpaTest, @WebMvcTest, @SpringBootTest — chaque slice teste ce qu'il faut, ni plus ni moins.



Isolation des données

@BeforeEach nettoie la base. Les transactions en @DataJpaTest font un rollback automatique.



Tester les erreurs

Chaque cas nominal a son cas d'erreur. 404, 400, 403 sont aussi importants que les 200.



Sécurité explicite

Tester chaque rôle sur chaque endpoint sensible. Ne jamais supposer que la sécurité fonctionne sans test.



Conclusion

Vous maîtrisez désormais les trois niveaux de tests Spring : **@DataJpaTest** pour le repository, **@WebMvcTest + Mockito** pour le contrôleur, **@SpringBootTest** pour l'intégration — et la sécurité avec **@WithMockUser**.

À pratiquer

Écrire les tests pour les endpoints de recherche paginée et les projections.

À explorer

TestContainers pour remplacer H2 par une vraie base PostgreSQL en intégration.

À retenir

Un test qui échoue est une information précieuse. Testez tôt, testez souvent.